

Programmation Java III

Rapport de projet : HELBDrone

Mr. Riggio

2025-2026

1 TABLE DES MATIERES

2	Introduction	3
3	Fonctionnalités de base	4
4	Analyse	9
5	Limitations.....	12
6	Conclusion	13
7	Annexe : Accès LLM.....	15

2 INTRODUCTION

Dans le cadre du cours de programmation Java III, il a été demandé à l'étudiant de réaliser un jeu de drones nommé HELBDrone. L'explication du jeu est la suivante : dans ce jeu, un pilote sentinelle est contrôlé par le joueur et doit se défendre contre une base aérienne générant continuellement des drones ennemis qui l'attaquent. Pour survivre, le joueur doit affronter et éliminer ces adversaires.

La réalisation de ce projet a pour objectif de respecter les principes de la programmation orientée objet, de garantir un code correct sans duplication, code mort, erreurs, variables magiques, etc., d'utiliser la bibliothèque JavaFX, d'appliquer le modèle Model-View-Controller, ainsi que la base du SnakeFX et la théorie vue au cours. Ce projet vise également à placer l'étudiant dans un contexte de relation entre chef de projet et client afin de favoriser l'apprentissage en vue d'une future carrière

Dans ce projet, de nombreuses fonctionnalités ont été implémentées. Tout d'abord, l'ensemble des éléments du jeu a été créé, notamment le pilote sentinelle, dont les déplacements ont été développés. Un nuage a également été ajouté afin de brouiller la visibilité du pilote, avec un système d'apparition et de déplacement. Des téléporteurs ont été intégrés pour permettre au pilote de se déplacer instantanément vers le côté opposé de la zone de jeu.

La base aérienne a été conçue avec un système de génération continue de drones, ainsi qu'un mécanisme basé sur un timestamp permettant de gérer les périodes d'invulnérabilité. Les drones, leurs déplacements, ainsi que les projectiles ont été implémentés. Divers éléments collectables ont été ajoutés, tels que la pierre philosophale ou le missile autoguidé, accompagnés d'un système de collecte fonctionnel. Les systèmes de tir, de gestion de la vie, de fin de partie et d'écriture du score dans un fichier ont également été réalisés, tout comme l'affichage du score dans le terminal.

D'autres composants complètent le projet, notamment l'intégration de codes de triche, la mise en place des niveaux, la gestion des points, l'architecture MVC, ainsi que la gestion de la vitesse, du temps et de l'espace.

Ce rapport sera présenté en plusieurs sections. La première section décrira les différentes fonctionnalités du projet. Elle sera suivie d'une partie « Analyse », expliquant la conception du code ainsi que les choix effectués, accompagnée de diagrammes. Une section « Limitations » proposera une autocritique de l'application, en abordant ce qui pourrait être amélioré avec davantage de temps ou les situations dans lesquelles l'application pourrait ne pas fonctionner correctement. Enfin, une conclusion viendra clôturer le rapport.

3 FONCTIONNALITÉS DE BASE

Dans cette partie, des fonctionnalités pertinentes offertes par le jeu seront présentées du point de vue technique et fonctionnel.

Tout d'abord, la fonctionnalité liée à la génération des drones est abordée. Les drones sont gérés par la base aérienne, qui dispose d'une *HashMap* destinée à stocker uniquement les drones actifs et encore en vie. De manière générale, de nouveaux drones sont générés lorsque la base aérienne devient vulnérable ou lorsqu'elle est touchée par un tir du pilote sentinelle. Le nombre maximal de drones pouvant être créés dépend de la manche en cours, ce qui permet d'ajuster progressivement le niveau de difficulté du jeu. De plus, le type de drone généré est sélectionné de manière aléatoire.

Sur l'illustration ci-dessous, il est possible d'observer que chaque drone créé est ajouté à la *HashMap* de la base aérienne ainsi qu'à la liste des éléments du jeu gérée par le contrôleur. Avant cette insertion, le type du drone est vérifié afin d'éviter la génération simultanée de plusieurs drones identiques.

```

77 // Génère des drones de manière aléatoire en fonction du niveau actuel.
78 // Utilise un nombre aléatoire pour sélectionner l'un des quatre types de drones à créer.
79 // Paramètres :
80 // - int round : nombre de drones à générer.
81 // - ArrayList<GameElement> gameElementList : liste dans laquelle les drones générés sont ajoutés
82 // afin que le contrôleur puisse gérer leurs interactions logiques.
83 // Retour : rien.
84 public void generateDrone(int round, ArrayList<GameElement> gameElementList)
85 {
86     // Permet d'éviter de générer un drone en plus lorsque la manche en cours est la dernière.
87     if(round > Controller.getMaxRounds())
88     {
89         round = Controller.getMaxRounds();
90     }
91
92     if(droneMap.size() != round)
93     {
94         String key = "";
95         int redDroneNumber = 0;
96         int orangeDroneNumber = 1;
97         int blueDroneNumber = 2;
98         int purpleDroneNumber = 3;
99         HashMap<Integer, String> droneTypeMap = new HashMap<Integer, String>();
100         droneTypeMap.put(redDroneNumber, RedDrone.RED_DRONE_KEY);
101         droneTypeMap.put(orangeDroneNumber, OrangeDrone.ORANGE_DRONE_KEY);
102         droneTypeMap.put(blueDroneNumber, BlueDrone.BLUE_DRONE_KEY);
103         droneTypeMap.put(purpleDroneNumber, PurpleDrone.PURPLE_DRONE_KEY);
104
105         // Le nombre de drones à générer se fait avec la manche en cours.
106         while (droneMap.size() < round ) {
107             int randomNum = (int)(Math.random() * Controller.getMaxRounds()); // Génération d'un nombre random.
108             // Vérification si type de drone n'est pas présent dans la hashMap.
109             // par la suite la base aérienne va créer un drone et l'ajouter dans la hashMap et dans la liste d'éléments de jeu.
110             key = droneTypeMap.get(randomNum);
111             if(!droneMap.containsKey(key))
112             {
113                 Drone drone = createDrone(key);
114                 droneMap.put(key, drone);
115                 gameElementList.add(CalculationConstants.MIN_INDEX, drone);
116             }
117         }
118     }
119 }

```

Le code présenté ci-dessous provient du fichier *AirBase.java*.

Par ailleurs, une seconde méthode de génération de drones est également implémentée. Elle reprend la même structure générale et le même nom, mais fait l'objet d'une surcharge grâce à un paramètre supplémentaire nommé *keyDrone*. Ce paramètre permet de créer un type de drone précis lorsqu'il n'est pas encore présent et lorsque la taille de la *HashMap* de la base aérienne n'atteint pas encore la limite imposée par la manche en cours (cheat code).

La seconde fonctionnalité abordée concerne le suivi de la cible par le missile autoguidé. Celui-ci est déclenché lorsque la touche *F* est pressée. Une fois consommé et lancé, le missile se dirige automatiquement vers la cible la plus proche, qu'il s'agisse d'un drone actif de la *HashMap* de la base aérienne ou de la base elle-même.

Le système de poursuite repose sur le même principe que le déplacement utilisé dans *SnakeFX_Automatic_to_Target*. À chaque appel de la méthode *moveToTarget*, la cible la plus proche est identifiée en fonction de la position actuelle du missile, puis ce dernier ajuste son déplacement case par case pour la rejoindre. Lorsque le missile atteint la position de sa cible, il inflige des dégâts et se détruit immédiatement.

Dans les images ci-dessous, il est possible d'observer, au travers du code, la représentation concrète des explications présentées.

```

117  @Override
118  public void performAction(Controller gameBoard){
119      // Vérification si le missile est active.
120      if(isGameElementFollowTarget())
121      {
122          // Déplacement du missile vers sa cible.
123          moveToTarget();
124          // Vérification du missile si il a touché sa cible.
125          gameBoard.checkMissileHit();
126      }
127  }

```

```

489  // Vérifie si le missile a touché une cible ennemie.
490  // Si c'est le cas, la cible subit des dégâts et le missile est détruit.
491  // Paramètres : aucun.
492  // Retour : rien.
493  public void checkMissileHit()
494  {
495      HashMap<String, Drone> targetMap = missile.getTargetMap();
496
497      // Vérification si le missile à toucher un drone.
498      for(Drone target : targetMap.values())
499      {
500          if(missile.getPosX() == target.getPosX() && missile.getPosY() == target.getPosY())
501          {
502              missile.triggerDamage(target);
503          }
504      }
505
506      // vérification si le missile à touché la base aérienne.
507      if(missile.getPosX() == airBase.getPosX() && (missile.getPosY() >= airBase.getPosY()
508      && missile.getPosY() < airBase.getPosY() + airBase.getHeightSize()))
509      {
510          if(airBase.isVulnerable())
511          {
512              missile.triggerDamage(airBase);
513              airBase.onAirBasehit(this, missile);
514          }
515      }
516  }

```

Le code illustré dans les images suivantes est extrait des fichiers *Missile.java* et *Controller.java*, dans lesquels les fonctionnalités sont implémentées.

Le comportement de déplacement des drones repose sur le même principe.

La troisième fonctionnalité concerne le système d'invulnérabilité temporaire du pilote sentinelle. Lorsque celui-ci consomme une pierre philosophale, l'état d'invulnérabilité est activé pour une durée limitée. Durant cette période, la couleur du pilote passe au rouge afin d'indiquer visuellement cet état au joueur, et aucun point de vie n'est perdu lorsqu'un projectile ennemi entre en collision avec lui.

Sur le plan technique, cette fonctionnalité repose sur un mécanisme basé sur le temps. Un timestamp est enregistré au moment de l'activation de l'invulnérabilité, puis comparé au temps actuel à chaque cycle d'exécution du jeu afin de déterminer si la durée autorisée est écoulée.

Sur les illustrations ci-dessous, il est possible d'observer le changement d'état du pilote sentinelle, la mise en place du délai d'invulnérabilité ainsi que la vérification permettant de déterminer la fin de cette période.

```

17  @Override
18  public void triggerAction(Controller gameBoard){
19      // La pierre philosophale disparaît après la consommation.
20      hide();
21      SentinelPilot sentinelPilot = gameBoard.getSentinelPilot();
22      // Le pilote sentinelle devient invulnérable.
23      sentinelPilot.setSentinelPilotInvulnerable(getTimeEffect(), gameBoard.getTeleporterList());
24      // Le pilote sentinelle devient rouge.
25      sentinelPilot.changeState();
26      // La pierre philosophale change de couleur pour avoir une nouvelle pierre philosophale pour la prochaine génération.
27      performAction(gameBoard);
28  }
29
30  // réfléchir comment faire le perform action.
31  @Override
32  public void performAction(Controller gameBoard){setPhilosopherStoneImage();}

```

```

143  // Rendra le pilote sentinelle invulnérable après la consommation de la pierre philosophale.
144  // Cette méthode prend en paramètre une liste de téléporteurs et les désactive.
145  // Paramètre :
146  // - List<Teleporter> teleporterList : la liste des téléporteurs à désactiver.
147  // Retour : rien.
148  public void setSentinelPilotInvulnerable(double invulnerabilityTimedays, ArrayList<Teleporter> teleporterList)
149  {
150      toggleVulnerable(); // Le pilote sentinelle devient invulnérable.
151      setInvulnerabilityTime(invulnerabilityTimedays); // Le timestamp est enregistré.
152      toggleHasEatenPhilosopherStone(); // Le pilote sentinelle a mangé une pierre philosophale.
153      for (Teleporter teleporter : teleporterList) // Les téléporteurs sont désactivés.
154      {
155          teleporter.toggleActive();
156      }
157      System.out.println("The pilot sentinel is invulnerable");
158  }

```

```

74  // on vérifie si le pilote sentinelle a consommé une pierre philosophale
75  // on vérifie par la suite si l'effet de invulnérabilité
76  // est terminé pour le remettre en vulnérable et aussi sa couleur initiale.
77  if(hasSentinelPilotAte())
78  {
79      if(checkPhilosopherStoneVulnerabilityTimeEffectEnd())
80      {
81          setSentinelPilotVulnerable(gameBoard.getTeleporterList());
82      }
83  }

```

Le code illustré dans les images suivantes est extrait des fichiers *PhilosopherStone.java* et *SentinelPilot.java*, dans lesquels les fonctionnalités sont implémentées.

Le principe du *timestamp* est également utilisé pour gérer les délais de tir ainsi que le changement de couleur des différents éléments vivants du jeu.

La quatrième fonctionnalité concerne le tir et la gestion des dégâts. Le joueur peut tirer manuellement en appuyant sur la flèche du haut, tandis que les drones tirent automatiquement. Pour éviter un tir en continu, chaque entité capable de tirer doit respecter un délai minimum entre deux projectiles.

Sur le plan technique, chaque élément du jeu pouvant tirer possède une liste dans laquelle sont enregistrés tous ses projectiles actifs. Ces listes permettent de suivre la position des projectiles, de les déplacer et de les supprimer lorsqu'ils touchent une cible ou sortent de la zone de jeu.

L'une des principales difficultés a consisté à gérer simultanément plusieurs projectiles provenant de différentes sources tout en assurant une détection fiable des collisions. Les projectiles peuvent toucher trois types de cibles : le pilote sentinelle, la base aérienne et les drones. À chaque cycle d'exécution du jeu, la position de chaque projectile est comparée à celle de sa cible, en prenant en compte la largeur de l'entité ainsi que son déplacement vertical. Lorsqu'une collision est détectée, la cible subit des dégâts et le projectile est immédiatement retiré de la liste des projectiles actifs.

Les illustrations ci-dessous montrent comment les projectiles sont gérés, leur cycle de vie et le mécanisme de détection des impacts, permettant ainsi de mieux comprendre cette fonctionnalité.

```

81 // Permet de déplacer les projectiles tirés par l'élément.
82 // Paramètres : aucun.
83 // Retour : rien.
84 public void generateProjectile(Controller gameBoard)
85 {
86     ArrayList<Projectile> projectileList = getProjectileList();
87     if(projectileList.size() > CalculationsConstants.MIN_SIZE)
88     {
89         for(Projectile projectile : projectileList)
90         {
91             projectile.performAction(gameBoard);
92         }
93     }
94 }

```

```

52 // Permet aux éléments de tirer des projectiles après un court délai
53 // et ajoute les projectiles à leur liste correspondante.
54 // Paramètres : aucun.
55 // Retour : rien.
56 public void shoot()
57 {
58     long currentTime = System.currentTimeMillis() / CalculationsConstants.MILLIS_DIVIDER;
59
60     // On vérifie si le temps actuel est plus grand que le temps du tir précédent.
61     if(currentTime > getShootingTime() + getshootingTimedelays())
62     {
63         // Ajoute le projectile dans la liste.
64         addProjectile(createProjectile());
65         // Fixe temps du nouveau tir.
66         setShootingTime(System.currentTimeMillis() / CalculationsConstants.MILLIS_DIVIDER);
67     }
68 }

```

```

111 // Supprime les projectiles du game element lorsqu'ils dépassent la zone limite
112 // ou lorsqu'ils touchent leur cible, afin d'optimiser les ressources.
113 // Paramètres : aucun.
114 // Retour : rien.
115 private void checkProjectileInProjectileList(Controller gameBoard)
116 {
117     ArrayList<Projectile> removeProjectilesList = new ArrayList<Projectile>();
118
119     // Tout les projectiles de la liste sont parcouru
120     for(Projectile projectile : projectileList)
121     {
122         // on vérifie si le projectile est tiré par un drone ou le pilote sentinelle
123         // si le projectile à touché une cible ou est sortie de la zone le projectile est ajouté
124         // dans une liste des projectiles qui doit être retiré.
125         if(projectile.isTheProjectileShootByDrone())
126         {
127             if(gameBoard.checkDroneProjectileOutside(projectile) || gameBoard.checkDronetHit(projectile))
128             {
129                 removeProjectilesList.add(projectile);
130             }
131         }
132         else
133         {
134             if(gameBoard.checkSentinelPilotProjectileOutside(projectile) || gameBoard.checkSentinelPilotHit(projectile))
135             {
136                 removeProjectilesList.add(projectile);
137             }
138         }
139     }
140     // les projectiles qui doivent être retiré sont retiré de la liste
141     removeProjectileInList(removeProjectilesList);
142 }

```

Le code illustré dans les images suivantes est extrait du fichier *Shooter.java*, dans lesquels les fonctionnalités sont implémentées.

La dernière fonctionnalité concerne la sauvegarde du score et du temps de jeu à la fin de chaque partie. Lorsque la partie se termine, le joueur doit saisir son nom dans le terminal afin que son score et la durée de sa session soient enregistrés. Le jeu affiche ensuite automatiquement le meilleur score ainsi que le temps le plus court parmi toutes les parties sauvegardées.

Sur le plan technique, cette fonctionnalité est entièrement gérée par la classe *ManageScore*. Une fois le nom du joueur récupéré, cette classe lit d'abord toutes les lignes du fichier de sauvegarde afin de les stocker dans une *ArrayList*. L'un des défis principaux a été d'insérer correctement un nouveau score au bon emplacement, tout en conservant une liste triée selon deux critères : le score le plus élevé et, en cas d'égalité, le temps de jeu le plus court.

Pour résoudre ce problème, un second parcours du fichier est effectué. Chaque ligne est analysée via un *split* permettant d'extraire les données du joueur concerné. Cela permet d'identifier précisément l'endroit où le nouveau score doit être inséré dans la liste.

Une fois la position déterminée, le nouveau score est ajouté dans l'*ArrayList*, puis la classe réécrit le fichier en respectant l'ordre mis à jour. Enfin, un dernier traitement permet de déterminer et d'afficher dans le terminal le meilleur score ainsi que le temps le plus court enregistrés.

L'illustration présentée ci-dessous montre, un exemple d'écriture de fichier en fin de partie.

```

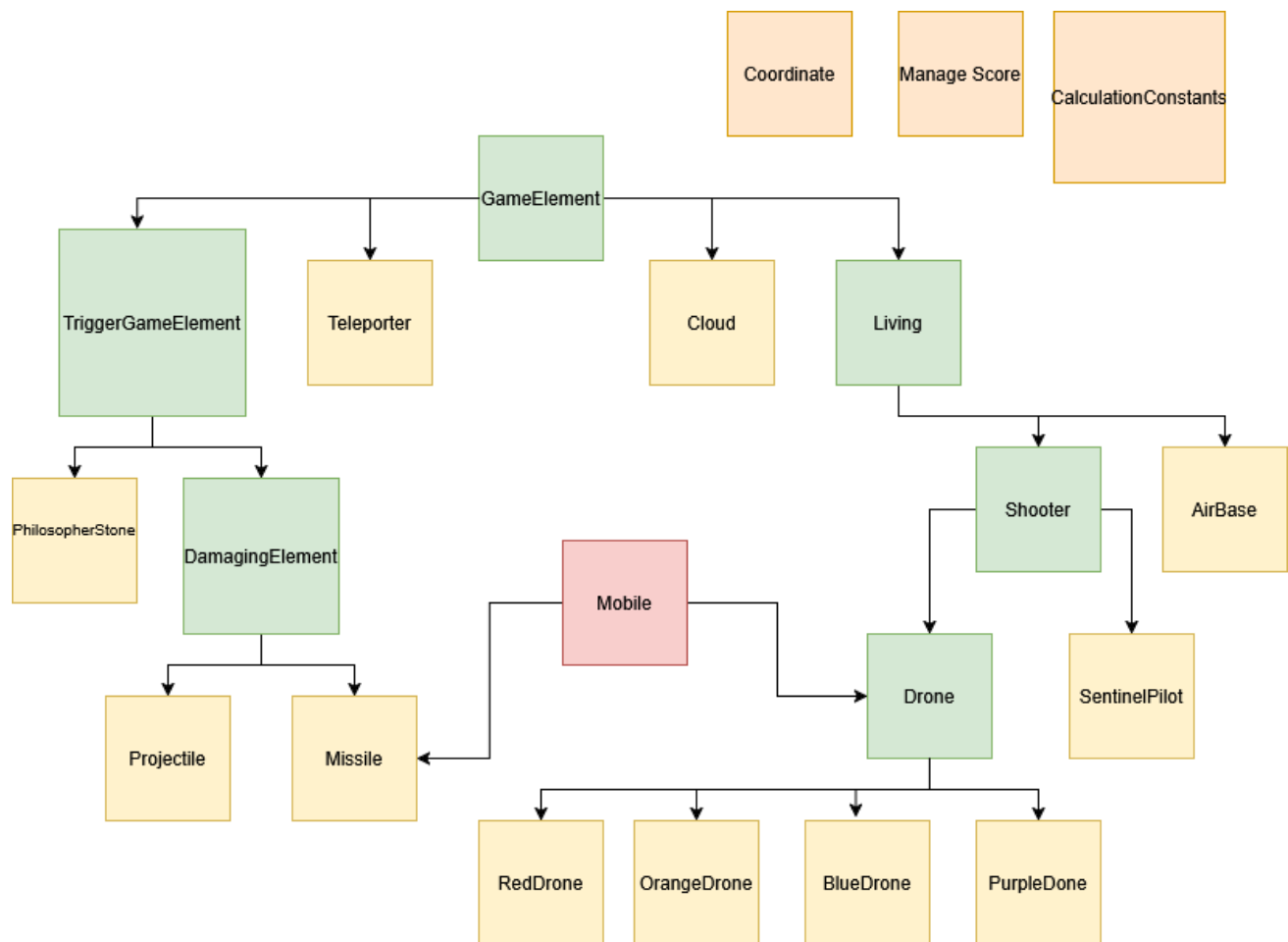
helb@helb-VirtualBox:~/Desktop/javaIII/MVC_HELB_DRONE/helbdrones$ bash run.sh
The pilot sentinel has self-destructed.
Enter username
Patrick
Username is: Patrick
Your score is 0
The highest score is Alice : 20
Successfully wrote to the file.

```

4 ANALYSE

Dans la section Analyse, la structure générale du code est exposée au moyen de schémas permettant d'explicitier les choix de conception. Un premier schéma est consacré à la modélisation et à l'héritage, afin de mettre en évidence la manière dont les classes ont été spécialisées et organisées au sein de la hiérarchie. Un second schéma présente quant à lui les relations établies entre les différents modèles et la façon dont ils interagissent au sein du système.

L'illustration ci-dessous présente le schéma relatif à l'architecture des modèles et aux décisions de conception qui y sont associées.



Dans le schéma présenté, un code couleur est utilisé afin de distinguer clairement les différentes catégories de modèles. Chaque couleur possède une signification précise : le vert représente les modèles abstraits, le jaune les modèles dérivés, le rouge les interfaces et l'orange les modèles ne participant à aucune relation d'héritage. Cette distinction visuelle facilite la lecture du diagramme et permet d'identifier rapidement le rôle structurel de chaque élément.

L'ensemble des éléments visibles du jeu hérite du modèle parent `GameElement`, comme le nuage (*Cloud*), les téléporteurs (*Teleporter*) ou encore le pilote sentinelle (*SentinelPilot*). Cette organisation permet de partager des caractéristiques communes entre les entités et garantit une conception cohérente, réutilisable et plus facile à maintenir.

Le schéma met également en évidence trois modèles qui ne dérivent pas de `GameElement` : `Coordinate`, `ManageScore` et `CalculationConstants`. Ces modèles ne représentent pas des entités visibles dans le jeu et interviennent uniquement dans la logique interne. `Coordinate` est chargé de la gestion des positions, tandis que `ManageScore` s'occupe de la gestion et de la sauvegarde des scores. Le modèle `CalculationConstants` centralise des valeurs constantes utilisées dans les calculs, ce qui améliore la lisibilité du code, facilite la maintenance et évite l'utilisation de constantes dites « magiques ».

La hiérarchie se poursuit avec le modèle abstrait `Living`, qui hérite de `GameElement` et introduit le système de points de vie pour certaines entités, telles que la base aérienne, le pilote sentinelle et les drones. Un second modèle abstrait, `Shooter`, hérite de `Living` et fournit le mécanisme de tir utilisé par les drones et le pilote sentinelle.

Le modèle abstrait `Drone` regroupe les comportements et caractéristiques communs à l'ensemble des drones. Cette structure facilite l'ajout de nouveaux types de drones tout en limitant la duplication de code.

Enfin, l'interface `Mobile` est utilisée pour attribuer la capacité de se déplacer vers une cible aux drones ainsi qu'au missile autoguidé. L'utilisation d'une interface permet de contourner les limites de l'héritage simple en Java, en offrant un comportement commun à plusieurs entités sans nécessiter l'héritage de plusieurs classes mères.

Un autre modèle abstrait, `TriggerElement`, hérite également de `GameElement`. Il fournit la fonctionnalité de déclenchement d'actions lors des collisions, notamment pour des éléments comme la pierre philosophale ou le missile, qui peuvent être collectés. Enfin, le modèle abstrait `DamagingElement`, héritant de `TriggerElement`, introduit le système de dégâts appliqué aux éléments vivants du jeu lorsqu'ils entrent en collision avec un projectile ou un missile autoguidé.

Plusieurs relations ont été identifiées pour montrer clairement comment les entités interagissent entre elles.

Le pilote sentinelle est directement lié aux projectiles : il peut en tirer, et ces projectiles sont donc générés par lui. Les drones fonctionnent de la même manière, puisqu'ils peuvent eux aussi tirer leurs propres projectiles.

La base aérienne occupe un rôle central, car elle est responsable de la création des drones. Ceux-ci dépendent donc directement de la base qui les produit.

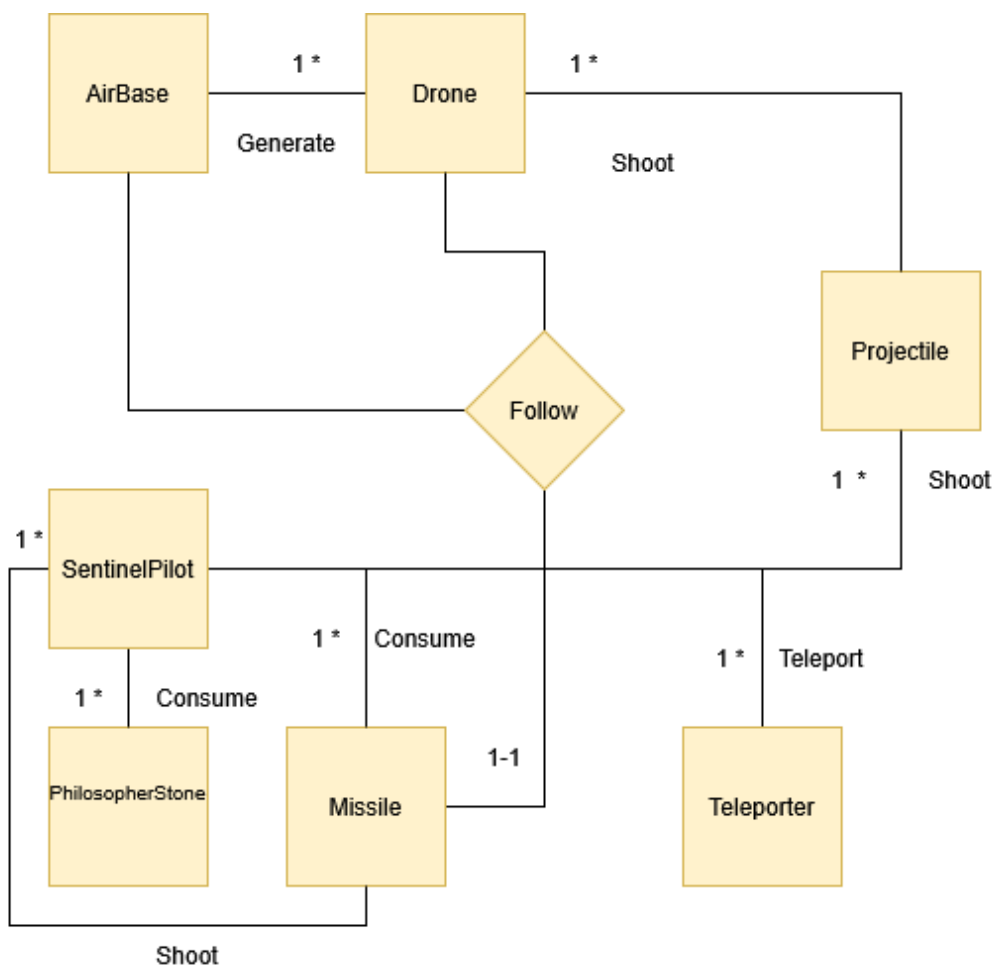
Le pilote sentinelle peut également consommer certains objets. Lorsqu'il consomme une pierre philosophale, il devient temporairement invulnérable. S'il consomme un missile, il peut alors activer le missile autoguidé, qu'il est aussi capable de tirer.

Les téléporteurs ont une interaction particulière avec le pilote sentinelle : ils le téléportent lorsqu'il entre en contact avec eux, ce qui crée une relation réciproque entre ces deux entités.

Le missile autoguidé peut poursuivre un drone ou la base aérienne, selon la cible disponible au moment de son activation. Ces entités peuvent donc être poursuivies par un ou plusieurs missiles.

L'ensemble de ces relations met en évidence la structure générale du jeu et montre comment les différents composants interagissent entre eux.

Le schéma ci-dessous illustre ces relations de manière visuelle. Il rassemble l'ensemble des interactions décrites et permet d'observer clairement comment chaque entité du jeu est liée aux autres.



5 LIMITATIONS

Plusieurs limitations ont été identifiées au cours de la réalisation de ce projet. Les points présentés ci-dessous constituent une autocritique constructive du travail effectué.

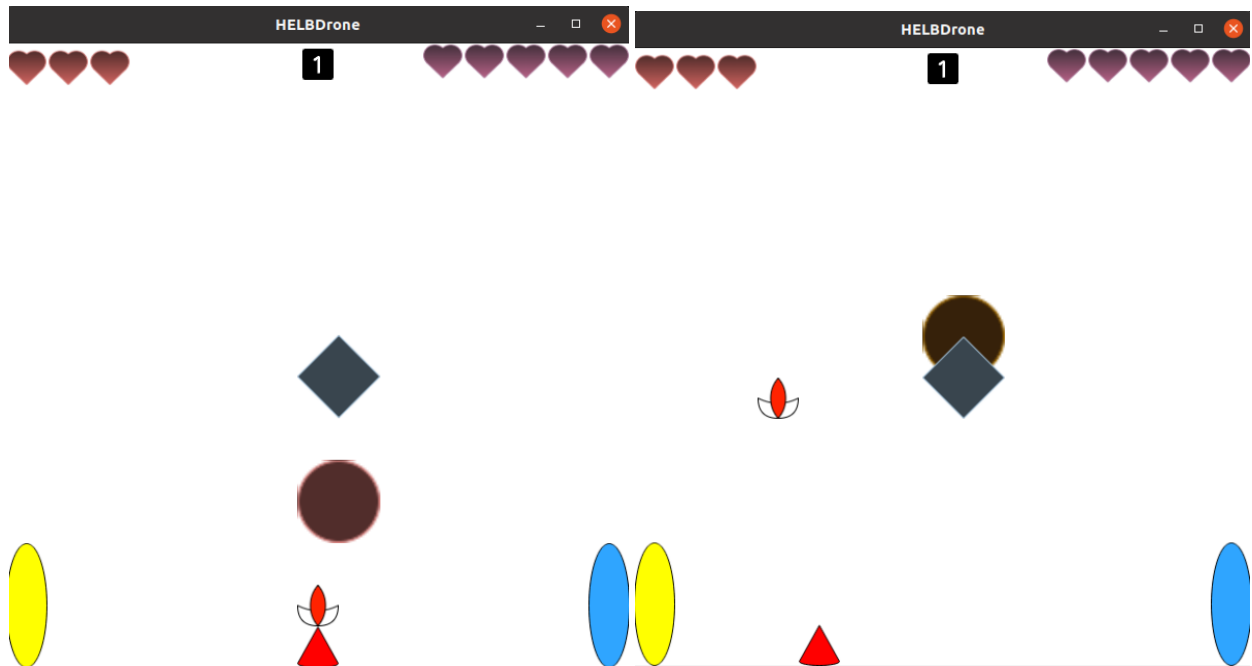
La première limitation concerne l'utilisation de listes pour la gestion des projectiles et des missiles. Ces structures sont utilisées pour déplacer les projectiles, vérifier les collisions, supprimer les éléments sortant de la zone de jeu et retirer les projectiles lorsqu'une collision est détectée, tout en appliquant les dégâts correspondants. Comme ces listes sont parcourues élément par élément, le temps de traitement augmente avec le nombre de projectiles actifs. Dans les niveaux plus avancés, cette approche peut entraîner des ralentissements visibles et affecter la fluidité du jeu.

Une seconde limitation concerne la détection des collisions. Celle-ci repose principalement sur la comparaison des positions en abscisse et en ordonnée des différentes entités. Cette méthode est simple à mettre en œuvre, mais elle devient moins fiable lorsque les éléments se déplacent rapidement ou lorsque leur taille varie. Afin de réduire ce problème, la taille des entités a été prise en compte lors des vérifications : les collisions sont détectées sur une zone correspondant aux dimensions des éléments et non plus sur un point unique. Cette amélioration permet une meilleure cohérence avec l'affichage visuel, même si elle reste moins précise qu'un véritable moteur de collisions.

Par ailleurs, le projet utilise des timestamps pour gérer différents délais, tels que le temps entre deux tirs, la durée d'invulnérabilité liée à la pierre philosophale ou le fonctionnement du missile autoguidé. Cette approche permet de contrôler les actions dépendantes du temps de manière simple. Toutefois, elle repose directement sur le temps du système, ce qui peut poser des problèmes à long terme, notamment en lien avec le bug de l'an 2038. De plus, l'ajout de nouvelles fonctionnalités temporelles nécessite d'introduire de nouvelles comparaisons de timestamps, ce qui complexifie progressivement le code et rend sa maintenance plus difficile.

Des limites ont également été observées dans le système de déplacement des drones. Lorsque la taille des drones est modifiée, des problèmes d'affichage peuvent apparaître, notamment lorsque ceux-ci semblent entrer visuellement en contact avec la base aérienne pendant leurs déplacements. De plus, la gestion des tirs des drones est impactée par ces changements de taille : les projectiles peuvent être générés depuis des positions incorrectes, comme les extrémités du drone. Ces comportements montrent que la prise en compte des dimensions réelles des entités reste incomplète dans certains calculs.

Les images présentées ci-dessous illustrent ces problèmes de déplacement et de tir des drones. Elles permettent de visualiser les incohérences observées lorsque la taille des entités est modifiée, tant au niveau de leur trajectoire que de la position d'apparition des projectiles.



Une autre limitation du projet concerne sa dépendance aux ressources matérielles. Le jeu devient plus fluide lorsque davantage de ressources sont disponibles, tandis que ses performances diminuent lorsque celles-ci sont limitées. Lors des tests réalisés dans une machine virtuelle Ubuntu, des ralentissements ont été observés lorsque peu de ressources étaient allouées. À l'inverse, en augmentant les ressources de la machine virtuelle, le jeu gagnait en fluidité. Cela montre que le bon fonctionnement du jeu dépend fortement des capacités de la machine utilisée.

Si davantage de temps avait été disponible pour ce projet, plusieurs améliorations auraient pu être mises en place afin de corriger les limitations identifiées. Il aurait notamment été possible d'optimiser les calculs effectués à chaque mise à jour du jeu, en particulier pour la gestion des projectiles, des collisions et des déplacements. L'utilisation de structures de données plus adaptées aurait permis de réduire la charge de calcul et d'améliorer les performances.

La gestion du temps, actuellement basée sur l'utilisation de timestamps pour les différentes fonctionnalités temporelles (temps entre deux tirs, durée d'invulnérabilité, effets temporaires liés à la pierre philosophale ou au missile autoguidé), aurait également pu être repensée afin de rendre le code plus simple, plus lisible et plus facile à maintenir. Enfin, une meilleure prise en compte de la taille réelle des entités dans les calculs de déplacement et de tir aurait permis de corriger les problèmes d'affichage observés.

6 CONCLUSION

Ce projet a permis de réaliser un jeu fonctionnel en appliquant les notions vues en cours. Les principaux éléments du jeu, tels que le pilote sentinelle, les drones, la base aérienne, les projectiles, les missiles, ainsi que le missile autoguidé, ont été implémentés avec leurs interactions. Le jeu est jouable et les mécanismes essentiels, comme les déplacements, les tirs, les collisions et l'utilisation de la pierre philosophale offrant un effet temporaire, fonctionnent correctement dans la majorité des situations.

L'utilisation du modèle MVC a contribué à une meilleure organisation du projet en séparant clairement la logique du jeu, l'affichage et le contrôle. Cette structure a permis de mieux comprendre le fonctionnement global de l'application et de rendre le code plus lisible et plus structuré.

Au cours du développement, plusieurs limitations ont été mises en évidence, notamment en termes de performances, de précision dans la détection des collisions, de gestion des déplacements et de gestion du temps à l'aide des timestamps. Ces difficultés ont permis de mieux comprendre l'impact des choix techniques et l'importance d'anticiper l'évolution d'un projet dès sa conception.

Dans l'ensemble, ce projet s'est révélé très formateur. Il a permis de renforcer la compréhension de la programmation orientée objet, du développement d'un jeu en temps réel et de l'organisation du code. Même si certaines améliorations auraient pu être apportées avec davantage de temps, le travail réalisé constitue une base solide et représente une expérience utile pour de futurs projets similaires.

7 ANNEXE : ACCÈS LLM

Un compte ChatGPT a été créé conformément aux consignes données par le professeur. Cet outil a été utilisé principalement pour aider à la rédaction du rapport, améliorer les commentaires des classes et des méthodes, et proposer des noms de variables plus explicites.

Son utilisation a été faite de manière ponctuelle et critique. Les propositions fournies ont été analysées et adaptées lorsque cela était nécessaire. ChatGPT n'a pas été utilisé pour générer automatiquement le code, mais comme un outil d'aide à la réflexion et à la clarification des idées.

L'identifiant du compte ainsi que les quatre dernières discussions sont fournis en annexe.

Identifiant du compte :

Email : ucar.ali.java.2526@gmail.com

Mot de passe : AliHelb1234!

Url des discussions :

<https://chatgpt.com/share/693f19d8-a734-8000-a899-591f94343199>

<https://chatgpt.com/share/693f19e9-bc64-8000-8dd2-c0c39e255d21>

<https://chatgpt.com/share/693f19fc-9348-8000-88d3-e80654068692>

<https://chatgpt.com/share/693f1a0f-55d4-8000-a9f9-45fcf8cd137b>